CLARITUNE
Making Things Happen

GDB debugging workshop for Mobileye

Alex Kushnir, Amir Kirsh, Adam Segoli Schubert

© All rights reserved.
Materials are for the sole use of the course participants, April-May 2026. Any other use is forbidden

About Claritune

Tailored Technical Courses:

- Modern C++ for Embedded Developers
- Secure Design and Development in C++
- Effective Unit Testing with GoogleTest
- C++ Multithreading and Concurrency
- Design Patterns for C++ Developers
- Linux Fundamentals
- DevOps: Docker
- DevOps: Kubernetes

and more ...

CLARITUNE
Making Things Happen

CTO: Amir Kirsh

Previously the Chief Programmer at Comverse.

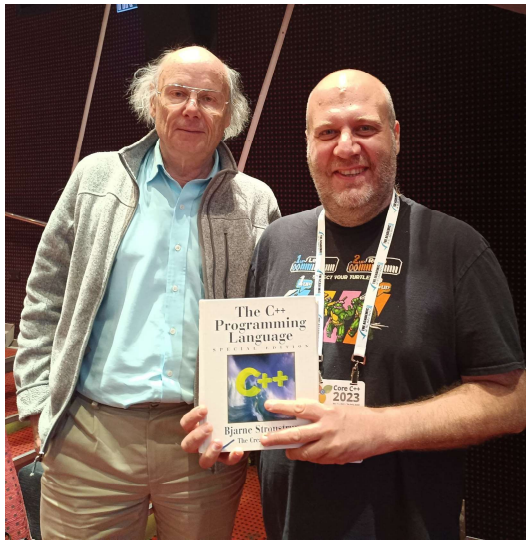
Lecturer at the Academic College of Tel-Aviv-Yaffo and Tel-Aviv University.

A team of top-level experts.

Trusted by a wide range of leading high-tech companies.

Who am I?

- Software engineer
- Mostly in the embedded and low-level domains
- Love C++
- Speaker at Core C++
  - Optimizing embedded software infrastructure
  - Mastering CTest
  - Managing dependencies with CMake
- Book reviews
  - Foreword for Modern CMake for C++
  - Editorial review for C++ for embedded systems
  - Post-release review for C++ memory management



GDB debugging workshop

Session 1

Introduction

Workshop agenda

Sessions 1 + 2

GDB basics

GDB TUI

Debug symbols

GDB in IDE

Advanced breakpoints

Sessions 3 + 4

Coredump analysis

Attaching to a process

Altering execution

Tracepoints

Checkpoints

Reverse execution

Remote debugging

Memory issues

Sessions 5 + 6

Multithreading

GDB customizations

Complementary tools

What's new in C++ standard?

From: “it crashed” to: “I know exactly why”

Well… debugging is not only for fixing crashes

Debugging is understanding program state, exact flow, wrong assumptions, race conditions and logical bugs.

The first rule of Debugging

... ?

THE FIRST RULE OF
DEBUGGING
IS _____.

The first rule of Debugging is _____

Debugging must be methodical!

A methodical debugging flow

1. **Ask the question:** Why is X 7 in flow Y?
2. **State the expectation:** X should be 9, because…
3. **Form a model:** What do I think is happening?
4. **Step through the flow:** Check each step against your model.

Methodical debugging approaches

1. **Divide and conquer:** Reproduce the bug in a smaller, simpler scenario. If it still happens, that's useful. If it doesn't, that's useful too.
2. **Record each run (open a doc for it!):** Note the scenario, whether the bug appeared, and any relevant observations.
3. **Change one thing at a time:** Avoid multiple edits in one iteration, so each run gives a clear signal. Document the change you made.

A methodical bug fix flow (1)

1. Do not fix a bug until you have a **solid theory** for its **root cause**.
2. If you have a theory about the root cause but you are not sure, **prove it first**, with a minimal external example.
3. Before pushing the fix, **add a unit test** that fails on the bug and passes with the fix.

A methodical bug fix flow (2)

4. **Keep commits isolated**; do not mix unrelated changes with the fix, (relevant logging if needed can be considered part of the fix).
5. **Require code review before merging**: explain the bug, the root cause, and why the fix is correct.

Tools for debugging

- Logs and printouts
- Statistics
- Debugger
- Other tools (valgrind, procdump, Wireshark, etc.)

Use GDB!

- for ***analyzing program flow*** – if you can't see the bug in a quick code review or in the logs, start a debugging session!

- for ***analyzing crashes and hangs***

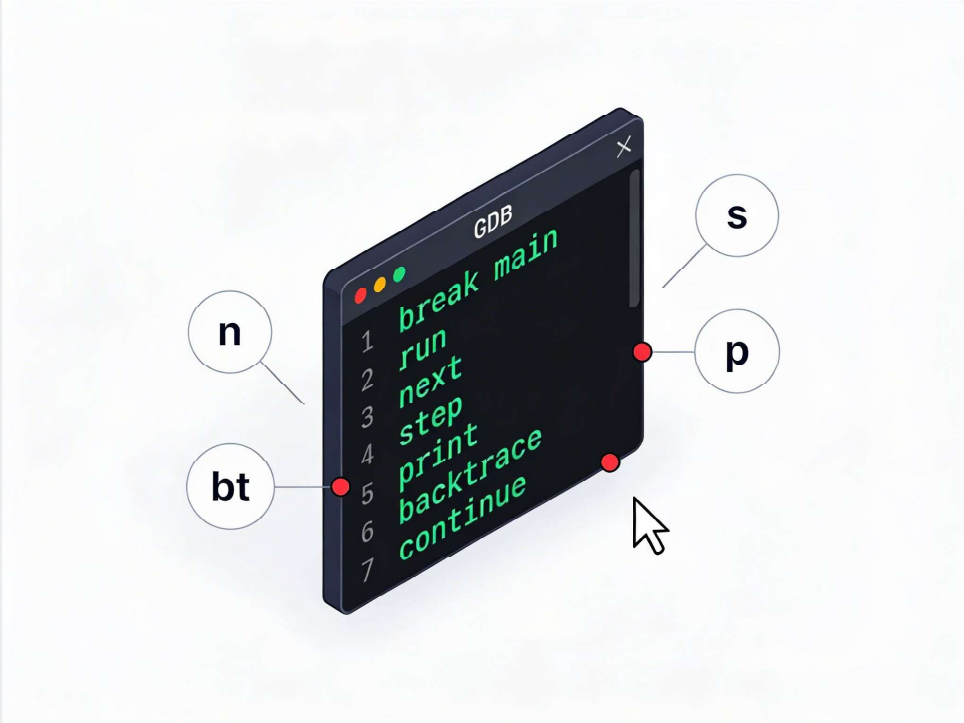
Debugging with GDB

Build with ***debug information***, reproduce the relevant flow, stop at the right places with ***breakpoints*** *or* ***watchpoints***, then ***inspect*** program state:

- stack frames
- variables, memory, registers
- ... and the exact point where behavior diverges from expectations.

GDB basics

Command line for the win!



Basic GDB usage demo

`day01/00_basic_example`

Let's start the GDB session by typing

```
gdb build/hello
```

Basic GDB commands

- **`list`** - displays source code lines around a specific location
- **`run`** - Run the loaded program
- **`start`** - Run the loaded program and stop at main
- **`continue`** - Resume program execution after a breakpoint
- **`next`** - Step-over
- **`step`** - Step-into
- **`finish`** - Step-out
- **`backtrace`** - Show call stack
- **`quit`** - Exit GDB

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Breakpoint hit

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Breakpoint hit

Step into the printHello() function

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Breakpoint hit

Step into the printHello() function

Step out of the printHello() function

Basic GDB usage demo - explained

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/hello...
(gdb) b main
Breakpoint 1 at 0x4811: file /home/alex/gdb_workshop/test/hello.cpp, line 5.
(gdb) run
Starting program: /home/alex/gdb_workshop/test/build/hello
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at /home/alex/gdb_workshop/test/hello.cpp:5
5               printHello();
(gdb) step
printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
6               std::println("Hello, world!");
(gdb) finish
Run till exit from #0  printHello () at /home/alex/gdb_workshop/test/printer.cpp:6
Hello, world!
Demonstration of finish command
main () at /home/alex/gdb_workshop/test/hello.cpp:6
6               return 0;
(gdb) cont
Continuing.
[Inferior 1 (process 26251) exited normally]
(gdb)
```

Set a breakpoint in main()

Run the program

Breakpoint hit

Step into the printHello() function

Step out of the printHello() function

Continue the program execution

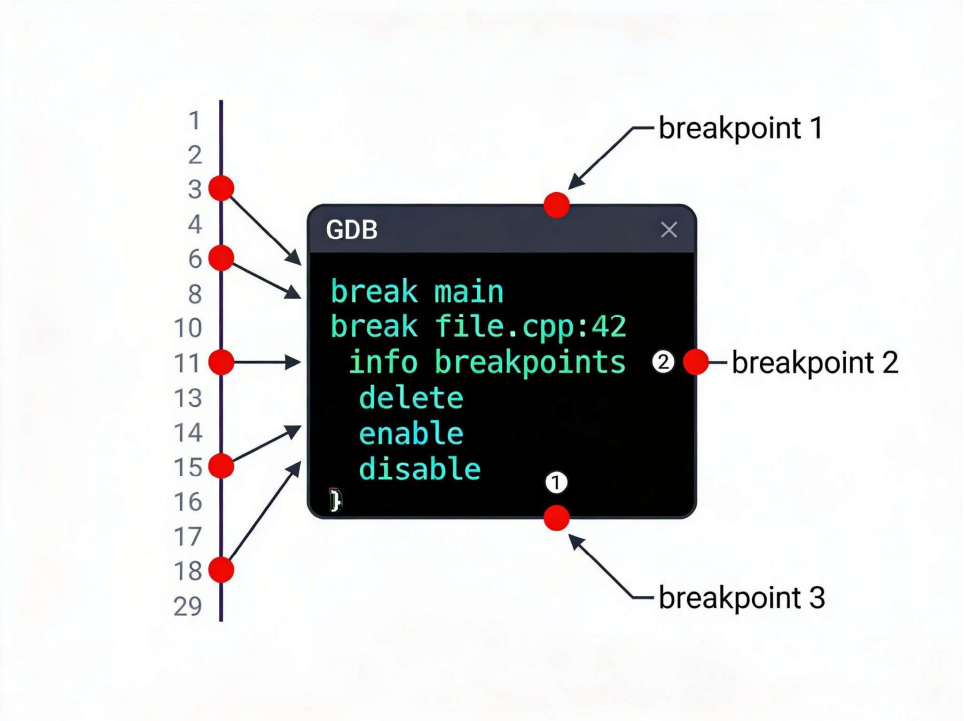
Providing arguments to the program

- Start debug session - **`gdb <program>`**
- Supply arguments in GDB CLI
  - **`run [args]`** - start the program with optional arguments
  - **`run < input.txt`** - provide stdin from file
  - **`run 2>errors.log`** - Redirect stderr to file
  - **`set args arg1 arg2 ... argN`** - Set arguments persistently
  - **`show args`** - Display current arguments

Debug information - The key for success!

- No debug symbols means no names, no lines, and barely any story - just addresses and guesswork.
- Debug symbols are the map from machine code back to source code: functions, variables, types, files, and line numbers.
- Make backtraces possible, because unwinding and frame reconstruction depend heavily on debug information such as DWARF.

Breakpoints



What Are Breakpoints?

- Pause the program execution at a specific location
- Allows the programmer to inspect memory, variables and call stack
- Several types of breakpoints exist
  - **Line breakpoint**: A breakpoint at a predefined line (and file)
  - **Conditional breakpoint**: A breakpoint that pauses execution only if a specific condition is met
  - **Data breakpoint (Watchpoint)**: Pauses when a specific memory address is read or modified

Setting Breakpoints

- **`break file.cpp:42`** - At line 42 in file.cpp
- **`break function_name`** - At function_name entry
- **`break Class::Method`** - Same as above, but in class member function
- **`break *0x00001234`** - At a specific function address (not variables!)

Managing breakpoints

- **`info breakpoints`** - List all breakpoints with IDs
- **`disable <breakpoint-id>`** - Disable breakpoint temporarily
- **`enable <breakpoint-id>`** - (Re–)enable the breakpoint
- **`delete <breakpoint-id>`** - Permanently remove the breakpoint
- **`clear file.cpp:42`** - Delete all breakpoints at line 42 of file.cpp
- **`set breakpoint pending on/off`** - Set in not yet loaded libraries

Breakpoints - Example

`day01/01_breakpoints/01_basic_example`

Managing breakpoints - commands

Actions on breakpoint hit (similar to “breakpoint actions” in some IDEs)

```
commands <breakpoint-id>
> <action1>
> ...
> <action N>
> end
```

Commands - Example

`day01/01_breakpoints/03_commands`

Breakpoints in overloaded functions

What happens when a breakpoint is set in an overloaded function?

Breakpoints in overloaded functions

What happens when a breakpoint is set in an overloaded function?

`break foo` ⇒ can be ambiguous, so GDB may find several matching locations.

GDB command **`set multiple-symbols <VALUE>`** can toggle the working mode.

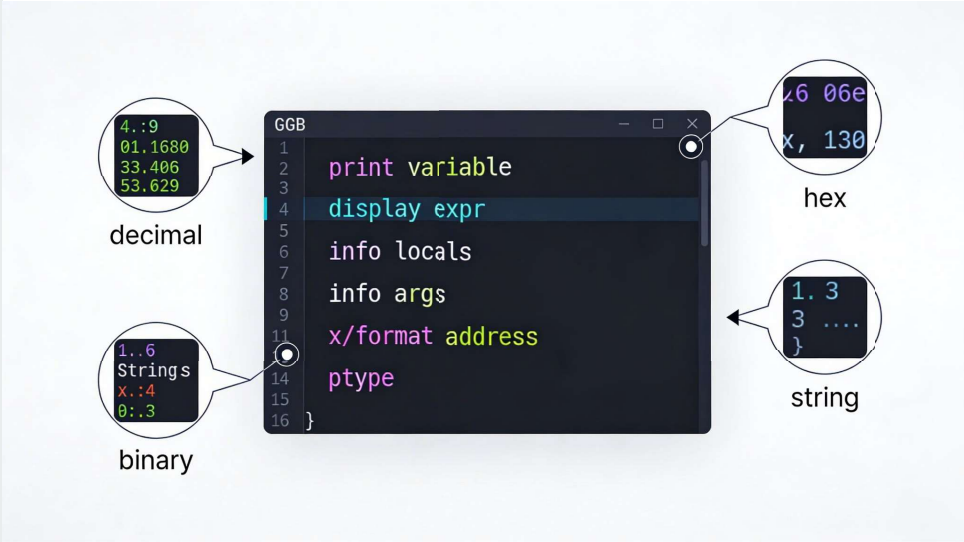
- **`all`** - set on all matches (default behavior)
- **`ask`** - GDB asks what to do
- **`cancel`** - Cancels the command if there is more than 1 match

The safest option is to name the exact overload, e.g. **`break foo(int)`**

Breakpoints - Example

`day01/01_breakpoints/02_functions_overload`

Data display in GDB



Why do we need to display data in GDB?

- **State verification at execution points** - Exposes unexpected state changes
- **Root cause identification** - Inspecting data helps trace errors to their source
- **Reduced time-to-resolution** - Data inspection eliminates guesswork and accelerates issue diagnosis
- **Pattern and anomaly detection** - Reveals recurring issues that are easily missed in raw text logs

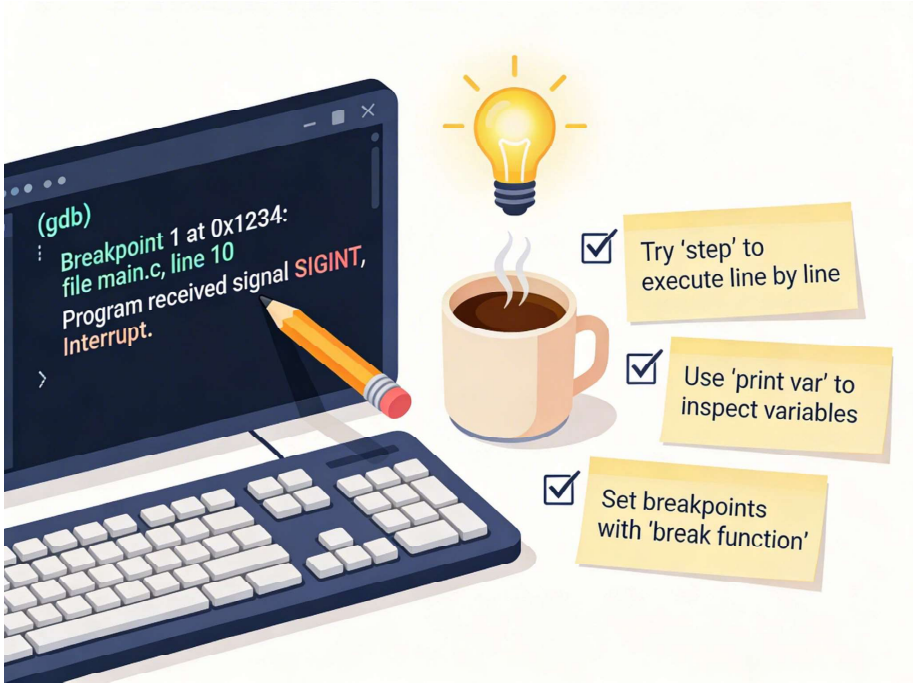
How to display?

- **`info locals`** - Show information about local variables
- **`info args`** - Show information about function arguments
- **`print variable`** - Show variable value (auto-formatted)
- **`print *ptr`** - Dereference pointer
- **`print *arr@N`** - Print array range (start at `arr` and print `N` entries)
- **`set print array on`** - Pretty-print arrays (one element per line)
- **`x 0x7fff1000`** - Examine memory at address (hex dump)

Some more display techniques

- **`display variable`** - Auto-print at each breakpoint
- **`info display`** - List active displays
- **`undisplay 1`** - Remove display #1
- **`whatis variable`** - Show variable type
- **`ptype MyClass`** - Show class structure and methods

Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

`exercises/day01/ex01/README.md`

GDB Cheatsheet

GDB cheatsheet - page 1

Running

`# gdb <program> [core dump]`
Start GDB (with optional core dump).

`# gdb --args <program> <args...>`
Start GDB and pass arguments

`# gdb --pid <pid>`
Start GDB and attach to process.

`set args <args...>`
Set arguments to pass to program to be debugged.

`run`
Run the program to be debugged.

`kill`
Kill the running program.

Breakpoints

`break <where>`
Set a new breakpoint.

`delete <breakpoint#>`
Remove a breakpoint.

`clear`
Delete all breakpoints.

`enable <breakpoint#>`
Enable a disabled breakpoint.

`disable <breakpoint#>`
Disable a breakpoint.

Watchpoints

`watch <where>`
Set a new watchpoint.

`delete/enable/disable <watchpoint#>`
Like breakpoints.

<where>

`function_name`
Break/watch the named function.

`line_number`
Break/watch the line number in the current source file.

`file:line_number`
Break/watch the line number in the named source file.

Conditions

`break/watch <where> if <condition>`
Break/watch at the given location if the condition is met.
Conditions may be almost any C expression that evaluate to true or false.

`condition <breakpoint#> <condition>`
Set/change the condition of an existing break- or watchpoint.

Examining the stack

`backtrace`
`where`
Show call stack.

`backtrace full`
`where full`
Show call stack, also print the local variables in each frame.

`frame <frame#>`
Select the stack frame to operate on.

Stepping

`step`
Go to next instruction (source line), diving into function.

`next`
Go to next instruction (source line) but don't dive into functions.

`finish`
Continue until the current function returns.

`continue`
Continue normal execution.

Variables and memory

`print/format <what>`
Print content of variable/memory location/register.

`display/format <what>`
Like „print", but print the information after each stepping instruction.

`undisplay <display#>`
Remove the „display" with the given number.

`enable display <display#>`
`disable display <display#>`
En- or disable the „display" with the given number.

`x/nfu <address>`
Print memory.
n: How many units to print (default 1).
f: Format character (like „print").
u: Unit.

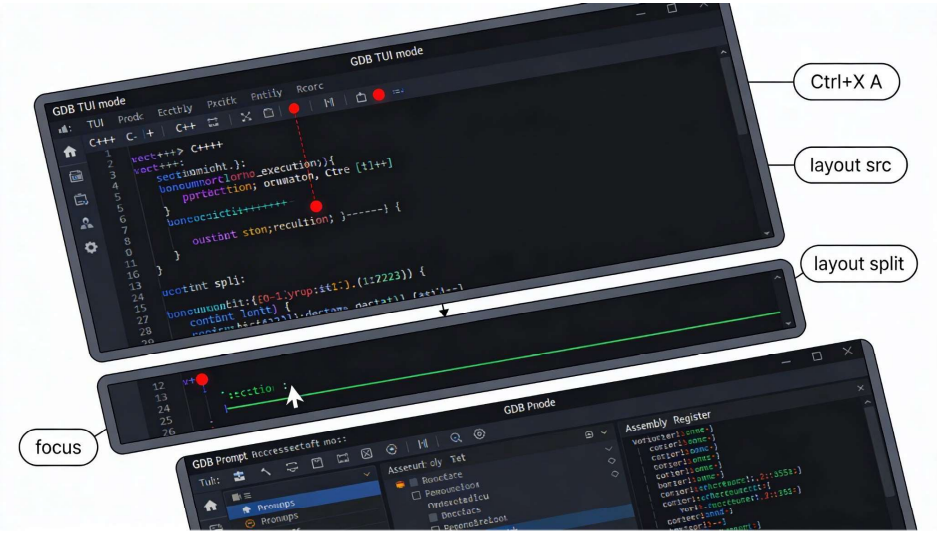
Unit is one of:

b: Byte,
h: Half-word (two bytes)
w: Word (four bytes)
g: Giant word (eight bytes)).

© 2007 Marc Haisenko <marc@darkdust.net>

https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf

Using GDB with TUI



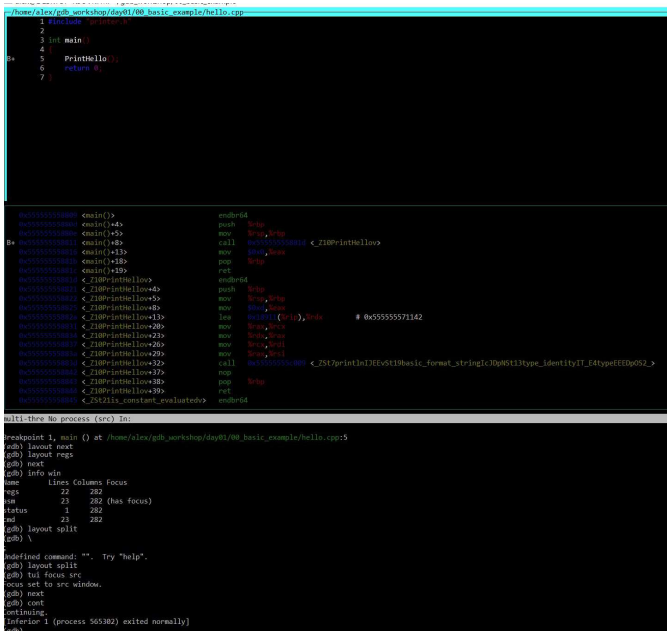
What is TUI? And how to use it?

TUI (Text User Interface) is a curses-based terminal view with windows for code, assembly, registers, and commands, making debugging more visual

What is TUI? And how to use it?

- **`gdb --tui <program>`** or **`tui enable`** from the **GDB** shell or **`Ctrl+X A`**
- Several layouts available
  - **`layout src, asm, regs`** - Displays the source code, disassembly and registers
  - **`layout prev/next`** - Cycle layouts
- **`tui focus <layout>`** - Focuses on the selected layout
- **`tui new-layout newlayout src 1 regs 1 status 0 cmd 1`** - Creates a new layout

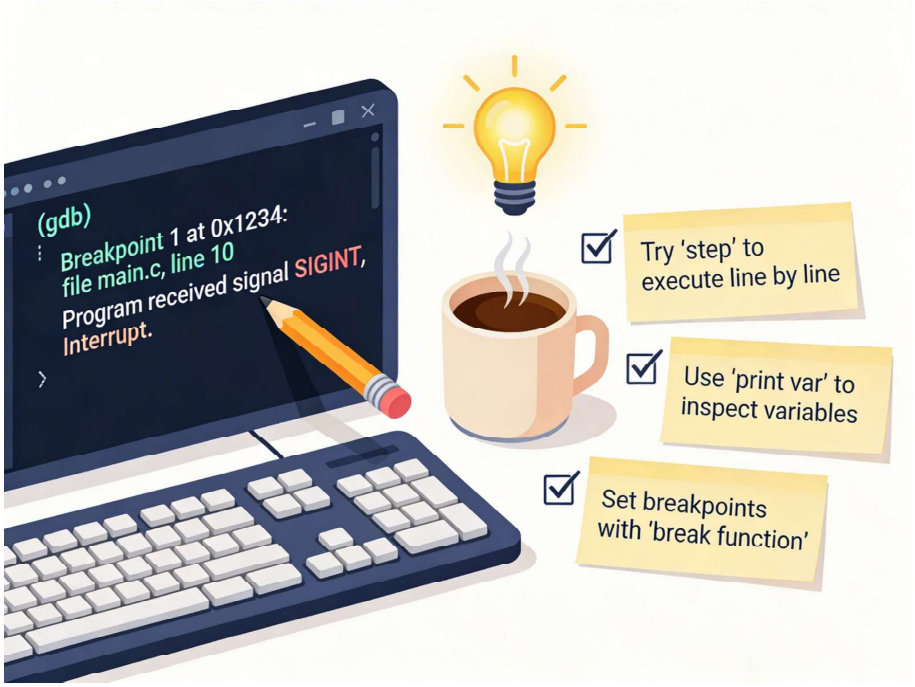
TUI layout example



GDB TUI - demo

day01/00_basic_example

Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

`exercises/day01/ex02/README.md`

Session 1 Summary

- Introduction
- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI

Towards Session 2

- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints
- Practical exercise